광운대학교 로봇게임단 로빛

HW01

TF2 를 내가 이해한 것을 바탕으로 설명 및 과제

학 번	2024402035
이 름	장경민
제 출 일	2026. 11. 05.

목차

1. 들어가며.....2

2. TF.....2

2.1 좌표계.....2

2.2 TF.....4

2.2.1 tf 의 관계(좌표계 간의 관계와)와 발행하는 법.....4

2.2.2 tf 수신하기 및 좌표계 변환.....5

2.2.3 tf 결론.....6

3. Launch.....6

3.1 Launch.....6

3.2 YAML 파라미터.....6

4. Action.....6

5. 실습.....7

5.1 TF2.....7

5.1.1 실행 결과.....7

5.1.2 간단한 고찰.....7

5.2 Launch.....8

5.2.1 실행결과.....8

5.3 Action.....8

5.3.1 실행 결과.....8

5.4 미니 과제 합쳐서 테스트.....9

5.4.1 Launch 로 실행 및 파라미터 조회.....10

5.4.2 Yaml 생성 및 적용.....10

5.4.3 실행화면.....10

1. 들어가며

이 과제는 오늘 배운 TF와 Launch, Action에 대해 이해한 대로 설명하고 간단한 과제를 실행한다. 설명하는 부분은 강의자료를 바탕으로 나의 약간의 배경지식과 인터넷 등으로 검색하여 이해한 부분을 작성할 계획이다.

AI를 이용해 찾은 부분도 많고 넘겨짚은 부분도 많아서 틀린 부분이 많을 수 있다.

2. TF

2.1 좌표계

TF는 좌표계간의 관계를 계산할 때 사용하는 프레임 워크이다. 이를 이해하기 위해서는 우선 좌표계에 관하여 알아야 한다. 좌표계는 원점과 수직인 축 3개로 구성된 것이다. 로봇에 들어가는 각각의 요소(LiDAR, 로봇팔, map 등)들은 서로 다른 원점과 축의 방향을 가지고 있다. 이들의 관계를 쉽게 계산하기 위해서는 하나의 좌표계의 포지션(상대 위치와 방향을 나타내고 싶은데 맞는 표현인지 모르겠음)을 다른 좌표계의 포지션으로 변환을 할 수 있어야 한다. 좌표계와 좌표계간의 관계를 계산할 때에는 2개의 요소가 필요하다. ❶ 축과 평행한 방향의 차이(이동) ❷ 축이 회전한 정도(회전). 이 둘을 표현하기 위해 여러 방법이 존재한다.

1. 평행이동 + 오일러 각

   오일러 각은 우리가 항공기등을 조종할 때 많이 사용하는 각으로 pitch, roll, yow 3개로 구성된다. 사람이 직관적으로 알아보기 쉬워서 많이 사용되지만 ros에서는 사용안한다. 그 이유는 각 각도의 적용순서에 따라 최종값이 차이가 나고, 특정각도에서 자유도를 하나 읽는 짐벌락 현상이 일어날 수 있기 때문이다. 최종적으로 평행이동과 오일러각을 이용한 관계 표현(연산은) 3(평행이동) + 3(회전)개의 값이 필요하다.

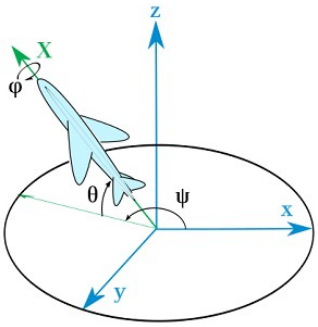


그림 1: 오일러각

2. 평행이동 + 쿼터니언(사원수)

   쿼터니언의 식 : $q = w + xi + yj + zk$

   위와 같은 문제를 해결하기 위해 ros2에서는 쿼터니언을 사용한다. 쿼터니언(사원수) 은 3개의 허수부와 1개의 실수부로 구성되어 있다. 3개의 허수부는 xi, yj, zk로써 회전 축과 회전 정도 나타내는 하나의 벡터를 그린다. 스칼라부는 w로 표현되며, 그 회전 축의 회전 정도를 나타낸다. 벡터부의 크기와 실수부의 크기의 비율은 회전축을 중심으로 회전하는 정도를 나타낸다. 벡터부의 크기는 축을 중심으로 회전하는 각도의 sin으로


그림 2: 짐벌락의 예시. pitch가 90deg가 되어 roll과 yow가 하나가 되었다

$|xi+yj+zk|=\sin(\frac{\theta}{2})$, 실수부의 크기는 축을 중심으로 회전하는 각도의 cos 으로 $|w|=\cos(\frac{\theta}{2})$ 계산하고 쿼터니언의 최종 크기는 1 이다. 따라서 벡터부와 실수부는 비선형적 반비례 (제곱의 합이 1)하는 크기가 반비례하는 성질이 있고 벡터부가 클수록 회전각이 크고, 벡터부가 작을수록 세타가 작다. 하나의 각도가 여러개의 값을 가질 수 있는것으로 알고있다. 최종적으로 평행이동과 쿼터니언을 이용한 관계 표현(연산)은 3(평행이동) + 4(회전) 개의 값이 필요하다.

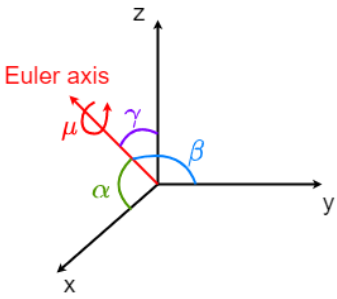


그림 3: 축-각도 표현법. 모든 회전은 1 개의 축과 그 축을 중심으로 하는 회전각도로 표현 가능하다. 이를 대수적으로 체계화 한 것이 쿼터니언이다.

3. 동차 행렬 (동차 좌표)

   동차 행렬은 위의 변환 방식들을 하나의 행렬로 나타낸 것이다. 평행이동과 회전은 각각 $n\times1$과 $n\times n$ 행렬로 나타낼 수 있다. 평행 이동 행렬과 회전 행렬을 하나로 묶어서 $(n+1)\times(n+1)$크기의 행렬로 나타낸다. 2D 에서는 3x3, 3D 에서는 4x4 크기이다. 이 동차 행렬을 이용해 연산을 하면 (부모)좌표계 - (자식)좌표계의 관계, (부모)좌표계에서 (자식)좌표계의 특정 점(위치), (부모)좌표계에서 (자식)좌표계가 틀어진 방향(회전정도)등을 매우 깔끔하게 구할 수 있다.

강의자료 7pg 에서는 앞에서 말한 이동 행렬, 회전행렬, 동차행렬등을 작성해 놓았고 8pg 에서는 쿼터니언을 간단히 작성해 놓았다. 이 것은 차후 tf 를 발행할 때에도 사용한다.



그림 4: 강의자료 8pg 에서 발췌

x-y 평면 만을 움직이는 거북이는 회전을 하면 z 축을 중심으로 회전하므로 위와 같이 나타낼 수 있다. 0deg 에서는 xyz 가 영벡터 이므로 회전이 없다. 90deg 에서는 z 축을 중심으로 90deg 회전한 것이므로 벡터의 방향은 z 방향, 크기는$0.707\approx\frac{1}{\sqrt{(2)}}$, w 의 크기는 $0.707\approx\frac{1}{\sqrt{(2)}}$이다. 이는 4 요소의 합은 1 이 됨을 만족한다.$\sqrt{(x^2+y^2+z^2+w^2)}=1$ 180deg 와 -90 또한 회전각의 sin, cos 관계, 벡터의 방향등을 만족한다.

여기서 한가지 의문이 들었다. 어짜피 xyz 벡터의 크기를 알면 w 를 구할 수 있고, 네 요소의 크기의 합이 1 이 되어야 한다는 것도 알고 있으니 굳이 w 를 사용할 필요 없이 xyz 벡터 하나만을 사용하여도 문제가 없지 않나 하는 것이다. AI 로 찾아보니 여러 문제가 있었다.

먼저 세타가 180deg 에 가까우면 정밀도가 무너질 수도 있다. xyz 크기를 이용해 w 를 역산하여 계산해야 하는데, 180deg 에 가까우면 xyz 가 1 에 가까워지기 때문이다. w 를 따로 저장하면 w 값이 정밀도를 보정해준다.

둘째로 로봇이 계속 회전하면 벡터가 순간적으로 반대방향을 향해야 하는 때가 온다. 여기서 불연속적인 부분이 나오기 때문에, (칼만)필터적용, 각속도를 위한 미분등을 계산하기 까다롭다. w 값을 사용하면 w 가 음수가 되므로 이 방식이 훨씬 더 좋다.

그 이외에도 몇가지 이유로 쿼터니언을 주로 사용하지만, 몇몇 부분에서는 3 개의 값으로 사용하는 것도 알게 되었다.(각속도 적분, ROS 에서 공분산 행렬, 위성자세제어등)

2.2 TF

2.2.1 tf 의 관계(좌표계 간의 관계와)와 발행하는 법

위와 같이 우리가 사용하는 다양한 요소들의 좌표계간의 관계를 알았으면 이제 실제 로봇에서 적용을 해야한다. 로봇에서 좌표계 간의 관계는 변화 여부에 따라 2 개로 분류할 수 있다.

1. Static TF
   Base_link <-> lider 와 같이 물리적으로 고정되어 있어 관계가 변하지 않는 좌표계 관의 관계에 사용한다. 토픽을 한번만 발행하고, static_transform_publisher 로 만든다.
2. Dynamic TF
   map <-> odom <-> base_link 와 같이 시간에 따라 동적으로 변하는 관계에 사용한다. 토픽을 계속 발행하여 주어야 하고, TransfromBroadcaster 로 발행한다.

좌표계 간의 관계는 트리구조로 작성되며 위의 두 방식 중 하나로 발행한 것이 TransfromListener 에 들어가면 관계가 형성된다.

/tf 토픽에는 TFMessage 하나만이 흐르는 데, 그 안에 transform 배열이 있다. 배열 내의 원소인 TransformStamped 에는 다음과 같은 항목이 들어있다. ❶ 토픽발행시각 ❷부모와 자식의 좌표계 id, ❸ 자식좌표계에서 측정한 좌표/벡터/좌표계등을 부모좌표계 기준 좌표/벡터/좌표계로 변환할 수 있는 평행이동벡터(3 개 요소)와 쿼터니언(4 개 요소).

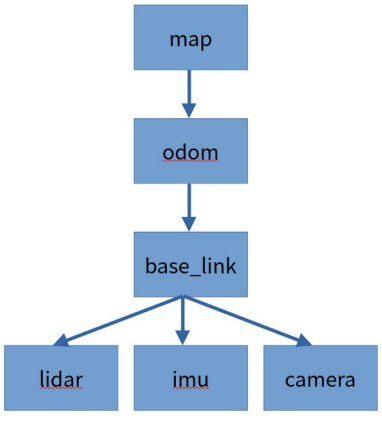


그림 5: 트리구조예제. 강의자료를 그대로 가져옴

```
transforms:
- header:
  stamp: {sec: 1730790960, nanosec: 112000000}
  frame_id: world
  child_frame_id: turtle1
  transform:
    translation: {x: 5.54, y: 5.54, z: 0.0}
```

```
    rotation: {x: 0.0, y: 0.0, z: 0.38, w: 0.92} # yaw ≈ 45°
```

tf 토픽 예시

두 좌표계의 관계를 아는 아무 노드에서 tf 토픽을 발행할 때 좌표계 간의 기하학적 관계를 사용자가 입력하여 주어야 한다.

```
class PoseBroadcaster(Node):
    def __init__(self):
        super().__init__('pose_broadcaster')
        self.name = self.declare_parameter('turtlename', 'turtle1').value
        self.br = TransformBroadcaster(self)
        self.create_subscription(Pose, f'/{self.name}/pose', self.cb, 10)
    def cb(self, msg): # 거북이 위치가 올 때마다
        t = TransformStamped()
        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id, t.child_frame_id = 'world', self.name
        t.transform.translation.x, t.transform.translation.y = msg.x, msg.y
        t.transform.rotation.z = math.sin(msg.theta / 2) # yaw → 쿼터니언
        t.transform.rotation.w = math.cos(msg.theta / 2)
        self.br.sendTransform(t)
```

위 코드는 강의자료에 나와있는 예제 코드로써 TransformBroadcaster 를 이용하여 Transform Stamped 를 발행한다. 이때 뒤에서 2,3,4 번 줄을 보면 좌표계간 관계를 기입하고 있고 cb 콜백 2, 3 번째 줄에서 시가과 부모/자식 좌표계의 이름을 넣고 있다.

이렇게 부모/자식 좌표계간의 관계를 완성시켜 놓으면 tf 수신을 통해 어느곳이든 원하는 좌표계간의 변환을 시킬 수 있다.

2.2.2 tf 수신하기 및 좌표계 변환

```
self.buf = Buffer(); self.lis = TransformListener(self.buf, self)
self.pub = self.create_publisher(Twist, '/turtle2/cmd_vel', 10)
self.create_timer(0.1, self.loop) # 10 Hz
def loop(self):
    try:
        t = self.buf.lookup_transform('turtle2', 'turtle1', rclpy.time.Time())
    except TransformException:
        return # 아직 TF 가 없으면 다음에
    x, y = t.transform.translation.x, t.transform.translation.y
    cmd = Twist() cmd.angular.z = 1.0 * math.atan2(y, x) # 상대 방향으로 회전
    cmd.linear.x = 0.5 * math.sqrt(x**2 + y**2) # 멀수록 빠르게
    self.pub.publish(cmd)
```

위 코드는 강의자료에 나와있는 예제코드이다. 버퍼를 만들고 lookup_transfrom 을 이용해 좌표계 간의 관계를 얻어 위치를 구하고 속도를 계산하여 발행하는 코드이다.

tf 를 이용하여 여러 곳에서 발행한 각각의 좌표계간의 관계는 TransformListener 함수를 이용하여 버퍼에서 받는다. lookup_transfrom 을 하면 버퍼에 저장해 놓은 tf 를 이용하여 두

좌표계 간의 관계를 도출할 수 있다. 예를들어 그림 6 과 예제코드에서 turtle2 의 좌표계를 기준으로 turtle1 의 좌표계를 계산하려면 (turtle2 의 좌표계에서 turtle1 의 위치. 왜냐하면 turtle1 의 좌표계의 원점은 turele1 의 원점일 테니까) lookup_transform('turtle2', 'turtle1',...)을 이용하여 구하는데, 이는 world -> turtle1 로 계산하는 동차행렬의 역행렬과 world -> turtle 로 계산하는 동차행렬의 곱으로 계산할 수 있다고 추측된다.



그림 6: 강의자료에 있는 그림. 거북이와 월드 간의 좌표계 관계가 적혀있다.

2.2.3 tf 결론

이렇게 tf 의 발행과 버퍼를 이용한 수신, lookup_transform 함수등을 이용하면 우리는 로봇에 사용하는 다양한 좌표계를 쉽게 변환할 수 있고 이를 이용해 로봇팔, 지도에서 목표지점 설정등 다양한 것을 만들 수 있다.

3. Launch

3.1 Launch

파이썬을 이용하여 launch 파일을 만들어 놓으면 여러 노드를 launch 파일 한번에 열 수 있다. 기존 방식은 ros2 run {패키지명} {노드명} 이렇게 노드를 하나씩 열어야 켜야 하였지만, launch 를 이용하면 ros2 launch {패키지명} {런치파일명} 미리 작성한 launch 파일을 이용해 여러 노드를 한번에 열 수 있다.

3.2 YAML 파라미터

yaml 파일을 만들어 놓고, launch 파일이나 노드등의 소스코드에 yaml 파일을 받는 코드를 작성해 놓으면 코드 수정 없이(새로 빌드 할 필요 없이) 설정등을 변경할 수 있다.

Yaml 파일에 /**, /{노드이름}등을 이용해서 적용범위를 설정할 수 있다.

4. Action

액션은 시간이 오래 걸리고, 중간 상항이 궁금하고 취소할 수 있는 일을 할 때 사용한다.

액션을 만들때에는 .action 이라는 목표, 결과, 진행사항이 들어있는 액션타입과 액션타입을 받아서 시행할 액션 서버, 액션을 요청할 액션 클라이언트가 필요하다. 터틀심에서는 액션 서버가 내장되어 있기 때문에 사용자는 액션 클라이언트를 만들어 사용하던가, 액션을 명령어로 주면 된다.

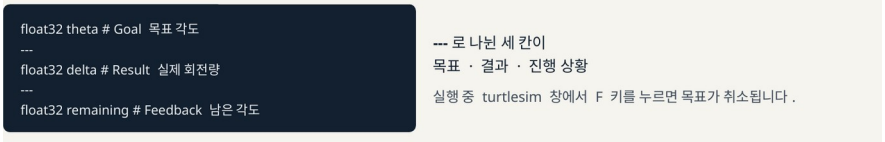


그림 7: 강의 자료에 있는 액션 타입

5. 실습

5.1 TF2

공식 데모를 실행시킨 화면이다. teleop 으로 turtle1 을 움직이면 turtle2 가 따라온다는 것을 확인하였다. 또한 view_frames 결과에 world → turtle1, world → turtle2 간의 트리 결과를 확인하였다.

5.1.1 실행 결과

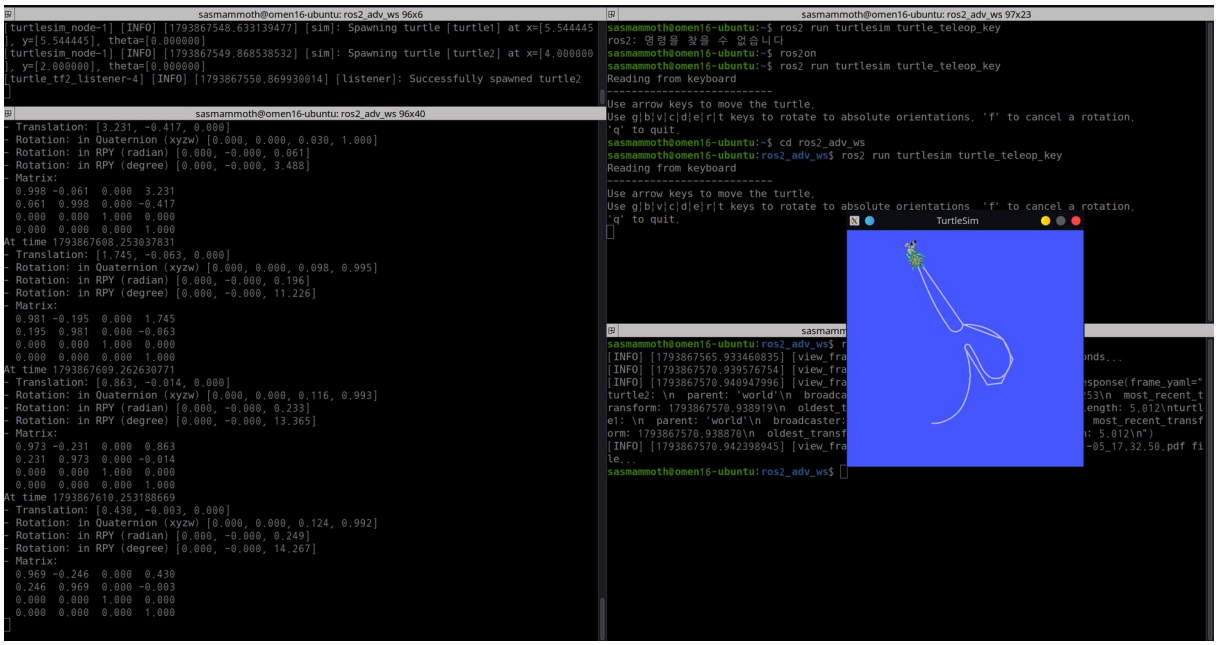


그림 8: 거북이가 따라가는 모습

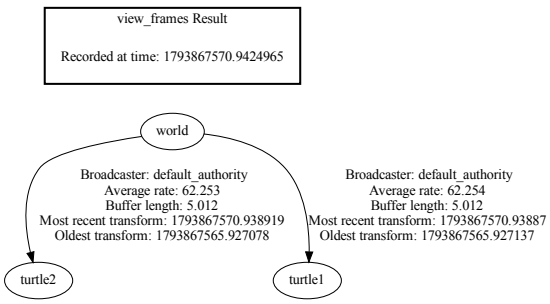


그림 9: 좌표계 간의 관계

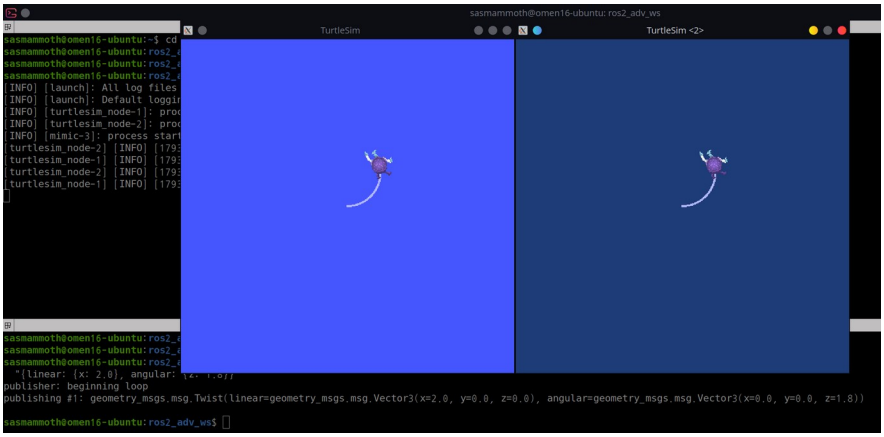
5.1.2 간단한 고찰

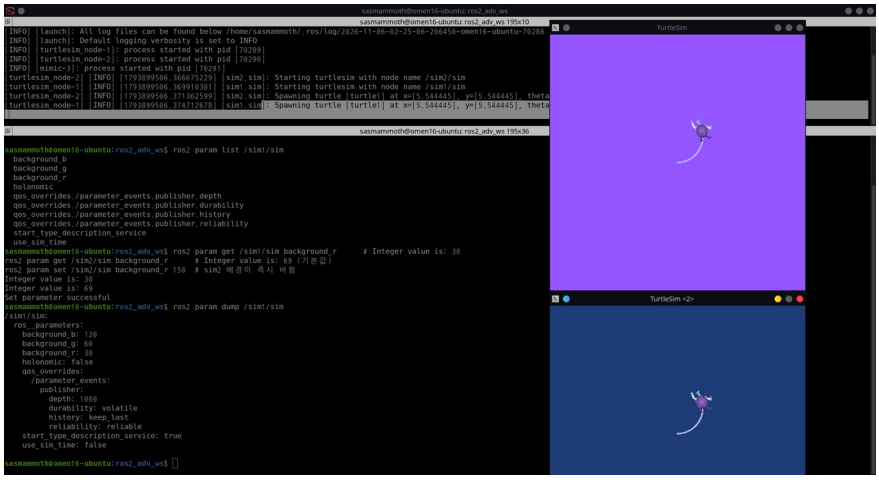
그림 8 은 거북이를 키보드로 움직인뒤 다른 거북이가 도달한지 얼마 안되었을때 찍은 사진이다. 우하단 로그를 보면 위치와 관련된 부분(4 열 123 행 or translation)이 시간이 지날수록 0 에 가까워 지고 있다. 이는 t1 이 t2 기준으로 어디에 있냐를 표시하는 부분이므로 (ros2 run tf2_ros tf2_echo turtle2 turtle1) 점점 줄어든다는 것은 점점 가까워 지고 있음을 뜻한다.

5.2 Launch

launch 를 이용해서 여러 노드를 한번에 실행시키고 ros2 param 으로 파라미터를 주입한다.

5.2.1 실행결과



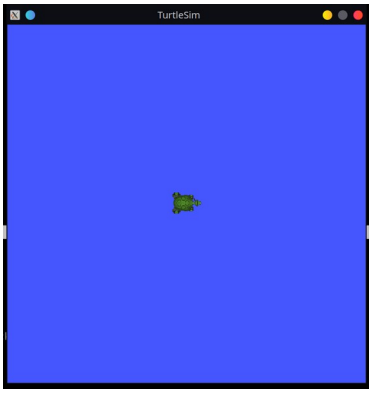


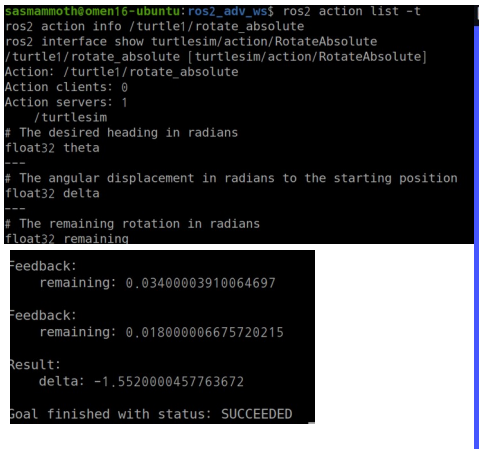
위의 그림은 launch 를 이용해서 여러개를 한번에 실행한 사진, 밑의 사진은 파라미터를 조회하고 색깔 파라미터를 주입하여 변경된 파라미터와 gui 를 출력한다.

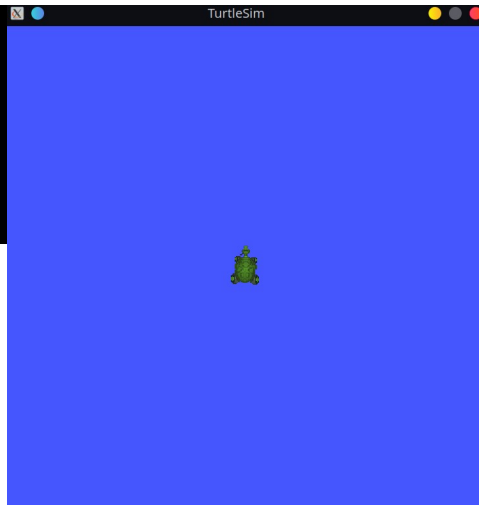
5.3 Action

action 을 이용하여 사용가능한 action 리스트와 정보를 조회하고 실제 동작을 확인한다. rotator 액션을 이용하여 turtle1 돌려도 turtle2 가 계속 따라온다는걸 확인하였다.

5.3.1 실행 결과

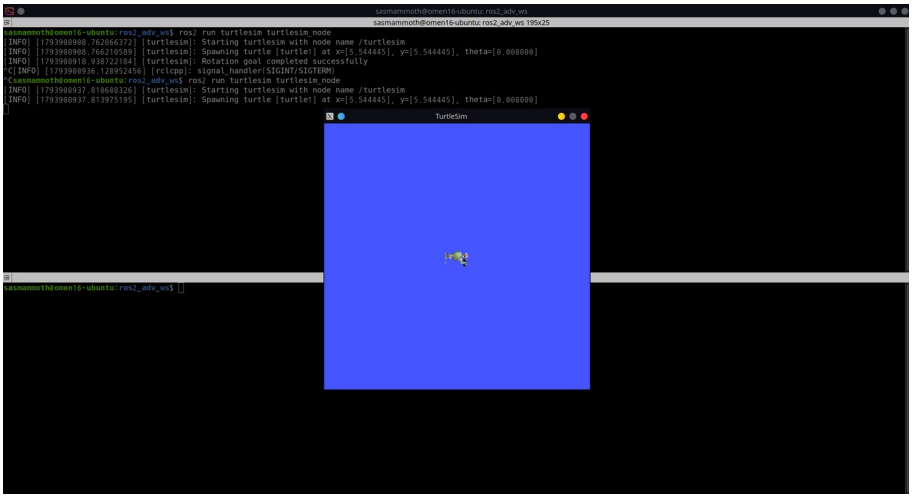


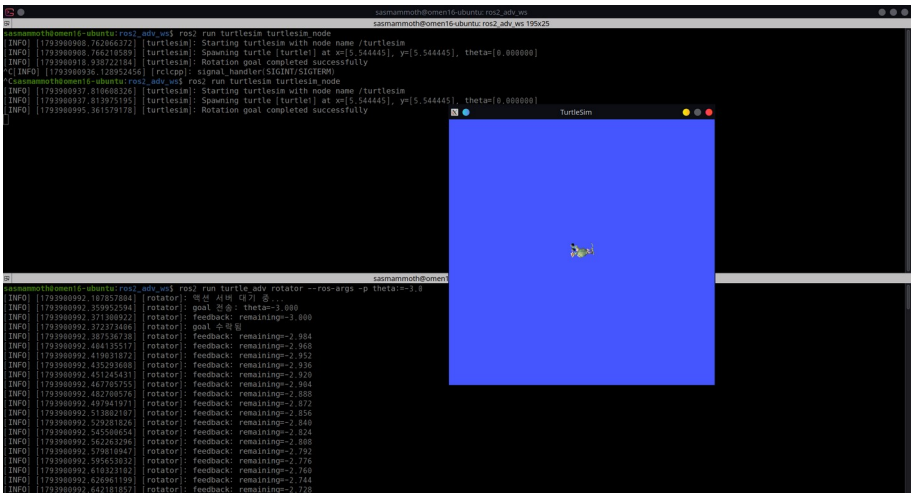


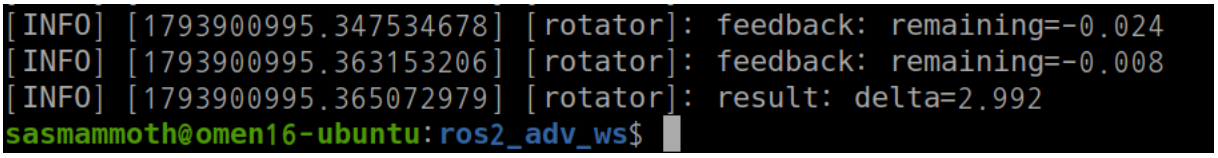


위의 사진과 같이 사용가능한 엑션을 조회하고 엑션 명령을 통해 거북이를 회전시켰다. 회전중에 실시간으로 남은 각도 피드벡이 왔다.

아래의 그림은 직접 만든 rotator 를 이용하여 엑션명령을 내려도 동작한다는 것을 보인다.

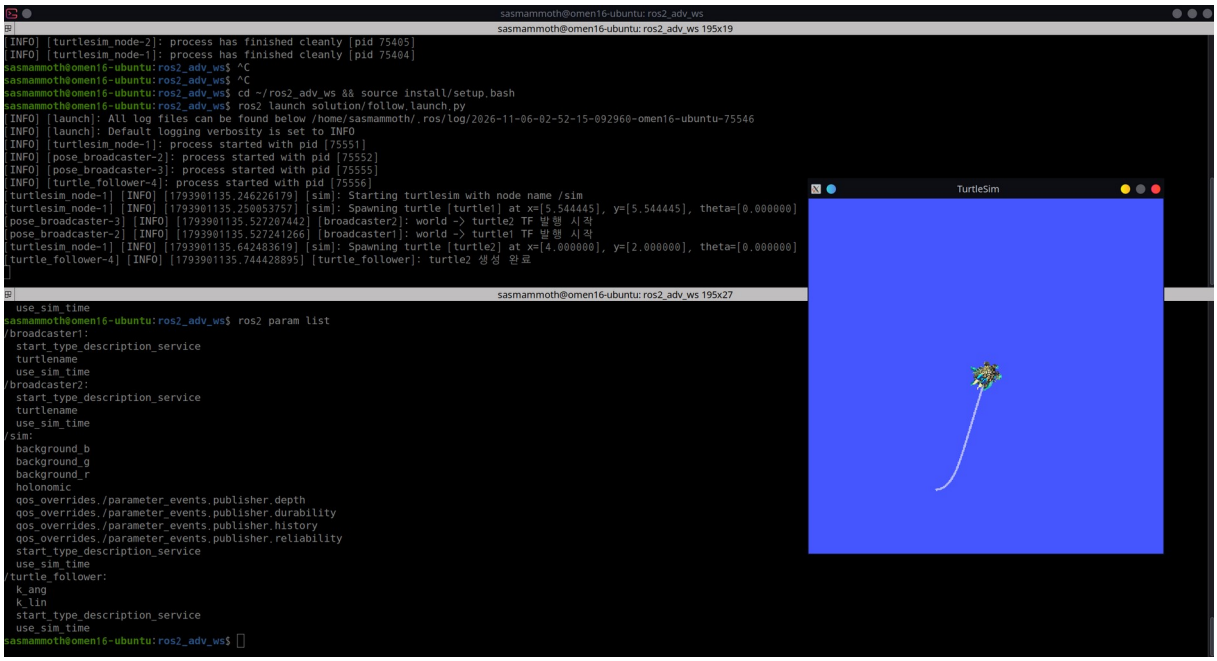






5.4 미니 과제 합쳐서 테스트

방법 A 로 실행 후 각각의 노드가 받을 수 있는 파라미터를 확인하고 YAML 에서 파라미터 속도 항목을 변환 후 적용하여 볼 것이다.



5.4.1 Launch 로 실행 및 파라미터 조회

```
ros2 launch solution/follow.launch.py
```

위 코드를 이용하여 실행 후 ros2 param list 로 따라가는 속도와 관련된 파라미터를 찾았다.

5.4.2 Yaml 생성 및 적용

이미 존재하는 ~/ros2_adv_ws/solution/follow_params.yaml 의 파라미터를 수정하여 따라오는 속도를 담당하는 k_lin 을 수정하여 실행한다.

5.4.3 실행화면

실행 영상 : https://drive.google.com/file/d/1QlyClP7MOTGkD5hsWxyOPMFovF9dy5DU/view?usp=drive_link

처음 k_lin 파라미터를 0.5 로 두고 실행하고 두 번째에는 3.0 으로 두고 실행하였다. 3.0 으로 하면 더 빠른 속도로 따라오는 것을 볼 수 있다. 처음에는 파라미터 폴더를 따로 두고 /turtle_follower 에 ros2 param load 를 이용하여 주입하려 하였다. 하지만 파라미터는 바뀌었는데 속도가 적용이 안되어 재시작 하는것으로 바꾸었다.